Comp 533 - Assignment 4: Object Replication Using RMI

Date Assigned: Sunday, March 09, 2025

Early Completion Date: Tuesday, Mar 25 2025, 11:55 pm (+5% extra credit)

Completion Date: Friday, Mar 28, 2025, 11:55 pm

Late Date: Tuesday, April 4, 2025, 11:55 pm (-10% late penalty)

Scary Late Date: Tuesday, Apr 11, 2025, 11:55 pm (-30 % late penalty)

In A3, you were exposed to RPC and used it in a Map-Reduce distributed algorithm. Now that we know this new form of procedure call, we are ready to experiment with a variety of distributed algorithms in A4-A6, which build on each other.

In this assignment, the focus will not be on a distributed implementation of a sequential algorithm, but object replication. The purpose of object replication here is to create a distributed collaborative user-interface like Google Docs or Zoom. However, replication serves other needs also such as fault tolerance and reduced latency.

Building a collaborative user-interface from scratch is difficult in a two-week project. Hence we have provided starter and scaffolding code to allow you to focus on distribution issues. The main challenge in this assignment is understanding the provided code and building a basis for the later assignments.

The code provided consists of four parts:

- A non-distributed single-user simulation to which you will add distribution to make it collaborative.
- A library for processing main arguments that specify connection information such as ports and hosts.
- A library for providing a command user-interface to provide console commands to manipulate the simulation and change distribution parameters.
- A library for tracing the distributed algorithm.
- A template that illustrates how the four components above can be used.

rea

The byte code of these four components is included in the 533 localchecks jar. The source of the template code is in: CoupledTrickOrTreat. The source of the three libraries is in the GitHub repository, GIPC. The source code and README file of the non-distributed program are project: AndersonBeau COMP401 Final Project. **You will need to look only at the template code.**

The existing non-distributed program is a Halloween simulation. You will create a distributed and replicated version of it using RMI remote procedure calls and the scaffolding code.

The non-distributed simulation is a project given to the Fall 2010, Comp 401 class (which was the precursor of Comp 301, taken after Comp 110, and before data structures, called Comp 410 at that time). You will work with the code of one of the students in the class – Beau Anderson (an Economics Major!) – who created a particularly nice simulation.

The assignment is explained in multiple ways. In addition to the initial description I created, I have included a visual way developed by Anna Wortas, an operational overview created by a student (Chris) in a previous class, and for the first time in this course, a GPT summarization. Multiple overviews make the assignment long, but hopefully clearer. The last part of it prepares you for the next two assignments, but is worth reading once you have this assignment in control.

A Visual Overview of the Assignment (Anna Wortas)

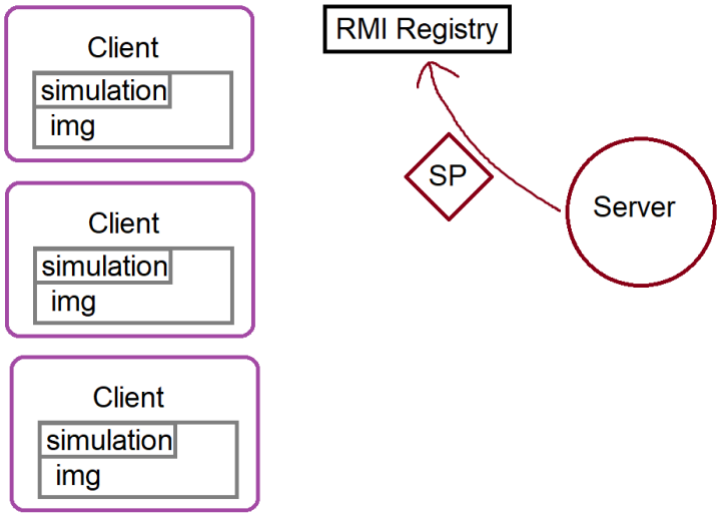
The basic idea in replication is to trap state changes (sent via property notifications) in one replica in a process and send them to replicas in other processes. If each replica is an observable, then essentially a local out-coupler traps state changes of an observable local replica, sends it to each remote in-coupler, which then changes the remote observable replica. This communication is done via a server that broadcasts changes from an out-coupler in one process to the in-couplers of all other processes.

Thus, to make this flow work, we need to know how to receive state changes to an observable replica and how to (programmatically) make state changes to the replica.

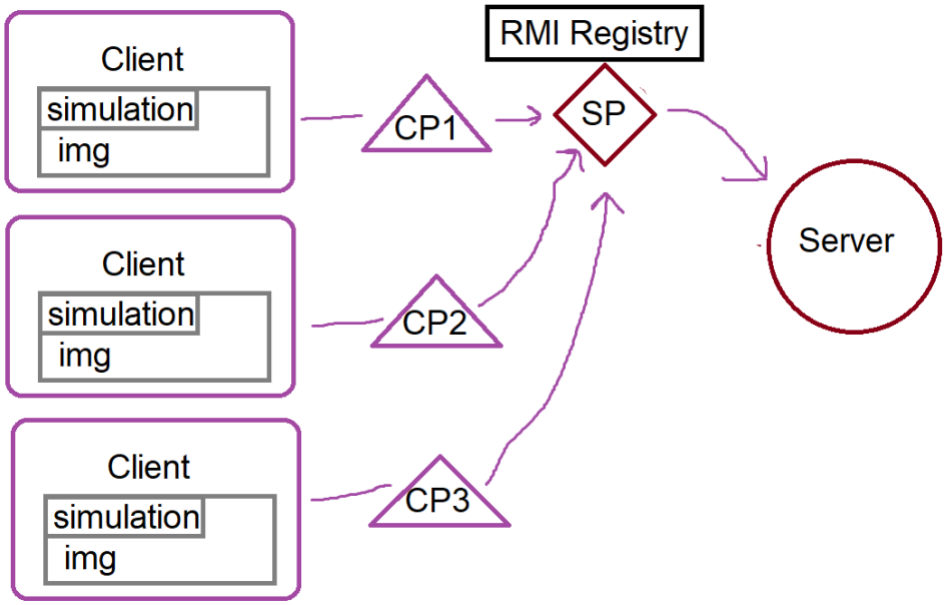
In this assignment, each observable replica will be a visual Halloween simulation described below. The exact nature of the replica is not important. A visual simulation of a well-known task should make testing of the system easier.

As in A3, you will have a server and multiple clients and will use RMI. This time the RMI registry will be a separate process outside of both the client and server. The clients will keep the replicas. The server will broadcast changes from one client to all of the other clients. To do the broadcast, the server will create its own problem-specific remote object that serves as a relayer remote object. Each client main class will register its in-couplers with the relayer. Each client out-coupler will send its changes to the relayer, which will then forward the change to other in-couplers registered with it.

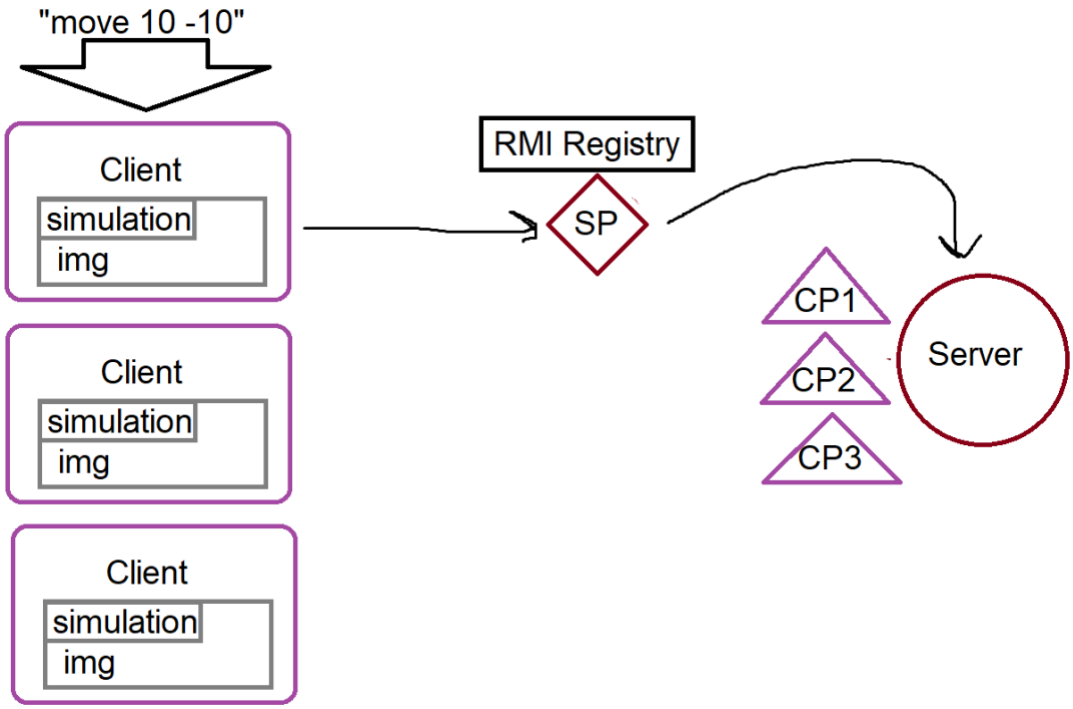
In the initial stage the server will access the RMI registry and add the relayer proxy to it.



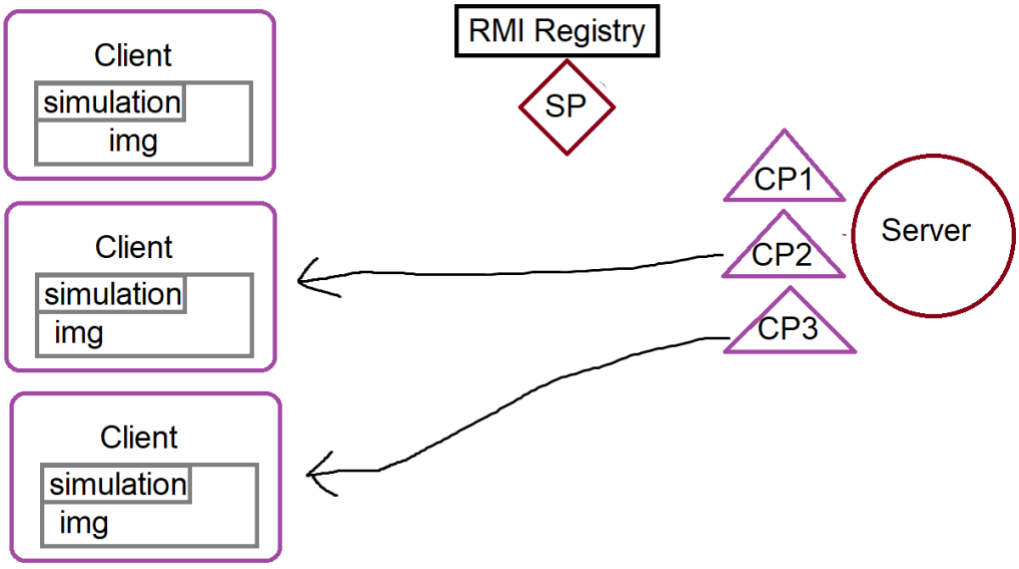
From here, the clients will access the RMI registry to find the server relayer proxy. They will then call a method in the server proxy to register their own in-coupler proxies to receive broadcasts.



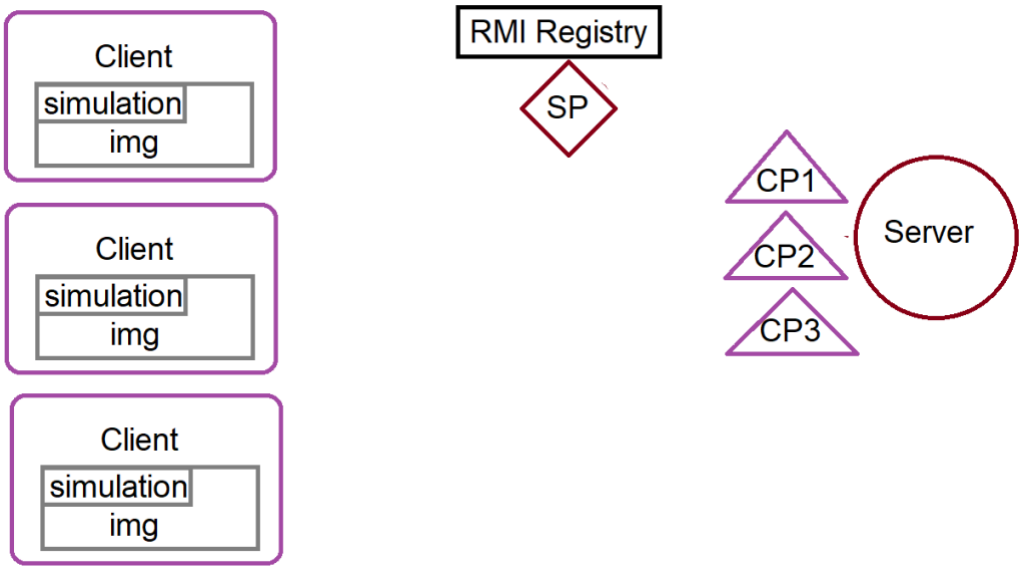
When new state-changing input is provided to one of the clients, it will update its own simulation state and use the server proxy to forward the message to the server.



The server will then propagate this message to all other clients excluding the one that initially sent the request by calling an appropriate method in each in-coupler.



The other clients that did not get the initial user input directly will then update their own simulation states to match the input received.



From here, a new round of inputs can start again.

Steps Overview

Here is a summary of steps you should take (details to follow):

1. *Non distributed program to be replicated*: Understand Beau's TrickOrTreat Halloween simulation project. This program creates observable simulation objects. In each replicated process, you will create an out coupler that listens to events of a local observable simulation object, and an in coupler that updates a local simulation object. As mentioned above, the in and out coupler objects in different processes communicate with each other through a server using RMI. That means, as in A3, you will need to support calls from a client to a server and callbacks from the server to clients. As in A3, a server can register its object in the RMI registry, but a client should not do so. It should call a method (defined by you) in the server object to register its remote in-coupler directly with the server so that the server has no knowledge of clients.
2. *Intra-process replication:* To understand how you can update simulation objects in out-couplers and receive events from simulation objects in in-couplers, the CoupledTrickOrTreat project is provided. It serves as a template for the client and server code you will write. This project shows how replicas of the simulation object can be created by *a single process*. Replicas are objects with the same state - a change to one replicas causes the corresponding change to be made in other replicas. Your goal will be to create replicas that live in *different processes*. You will have to create versions of the main class for the client and server.
3. *Inter-process replication*: Combine and extend the code above to support inter-process replication.
4. In this assignment, the RMI registry will be a separate process. In A3 the registry was combined with the server. The RMI PPTs show how such a registry is created and used in client and server programs.

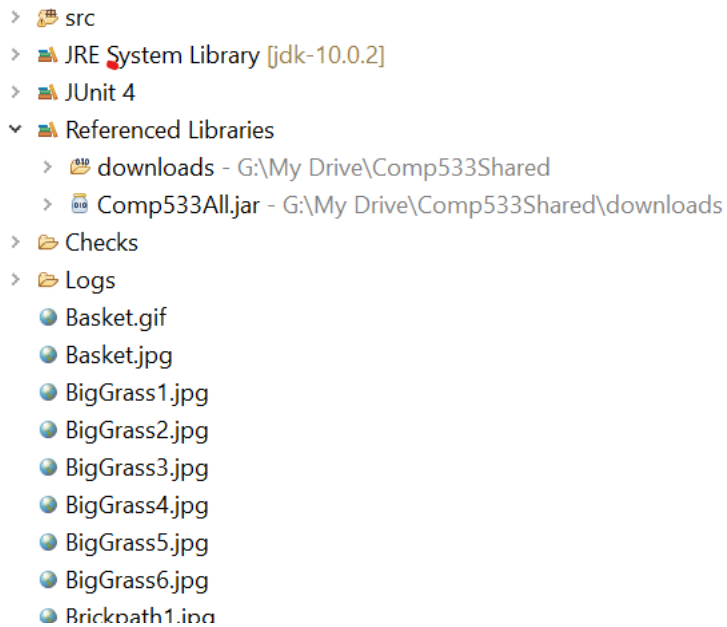
This assignment first describes the Halloween simulation you are turning into a distributed program. Next, it describes what you will be implementing. It is very important to have an understanding of the necessary aspects of the Halloween simulation model so you are aware of how it works as we will be using it for the remainder of the assignments in this course.

Non-Distributed Halloween (Trick or Treat) Simulation Details

Setting up the Project

The non-distributed simulation is provided in the 533 localchecks jar. It is best to understand it while following the steps below, otherwise it will appear to be more complicated than it is.

To run it, *create a new project*, and include the 533 localchecks jar in the classpath. Beau's code uses a large number of images. These can be retrieved from TrickOrTreatImages.zip Unzip the file and copy the images to your project folder (not the bin or src subfolders in it). After copying, you should see them in an Eclipse project as follows:



Running the Non-Distributed Simulation

You are now ready to run his code. As mentioned earlier, the source code and README file are in the git project:AndersonBeau COMP401 Final Project. You should not need to look at the source code unless you are curious. The contents of the README file are reproduced here.

Create in your project (in the src folder of course) a main class that calls Beau's main class and run it, as shown in the code below:

```java
package comp533;
import main.BeauAndersonFinalProject;

public class Main {

    public static void main(String[] args) {
        BeauAndersonFinalProject.main(args);
    }
}
```

The class creates two windows, a command window, and a graphics window. The command window is used to manipulate the objects shown in the graphics window.

The Take, Move and Give Commands

As shown in Figure 1, each graphics window consists of (a) one or more houses, each with a path and a candy container; and (b) a movable avatar with a candy container. The move command can be used to move the avatar in both the x and y directions, as shown in Figure 1 and reproduced below:

move 50 -50

If the feet of the avatar are in the path of a house, then the *take* and *give* commands can be used to transfer candies between the house and avatar containers.

For instance

take 3

takes tree candies from the basket, and

give 2

returns two of them.

These are the three commands you really need to know.

Other Commands for the Curious

For the curious, additional commands are provided to add and remove a house in the simulation, and undo or redo previous commands. The following is the syntax of the commands:

<Command> → <Move Command> | <Add Command> | <Remove Command> | <Take Command> | <Give Command> | <Undo Command> | <Redo Command>

<Move Command> → move <number> <number> (e.g. move 50 -50 // move 50 in x direction, -50 in Y direction)

<Take Command> → take <number> (e.g. take 2 // take two candies)

<Give Command> → give <number> (e.g. give 2 // return two candies)

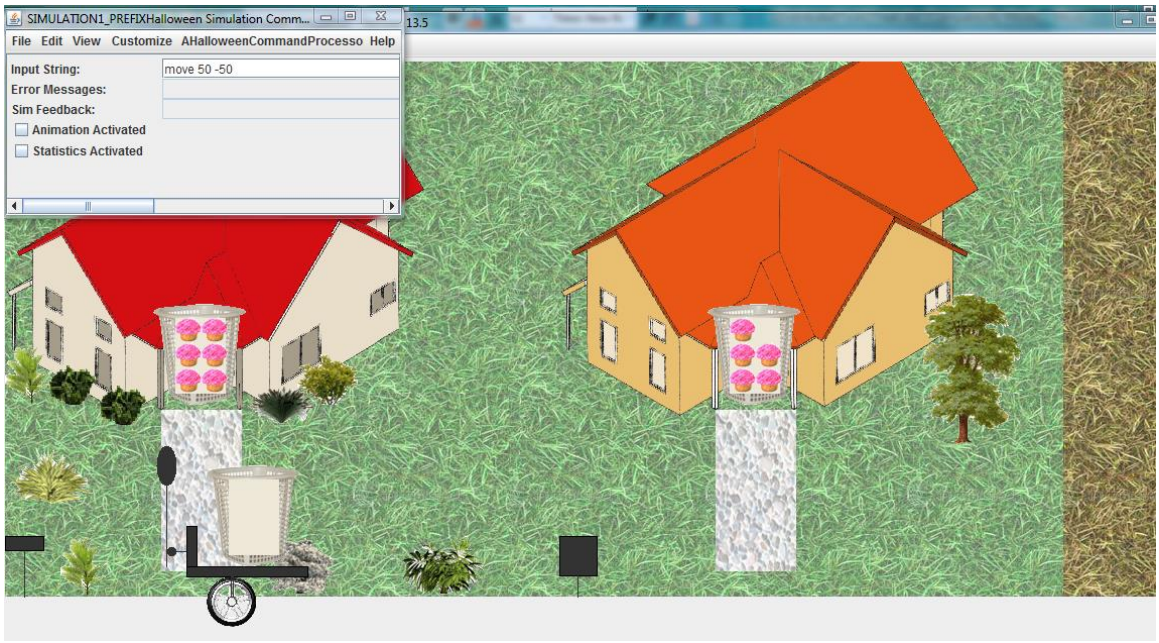
<Add Command> → addHouse (e.g. addHouse // add a new house to the simulation)

<Remove Command> → removeHouse (e.g. removeHouse // remove the last added house)

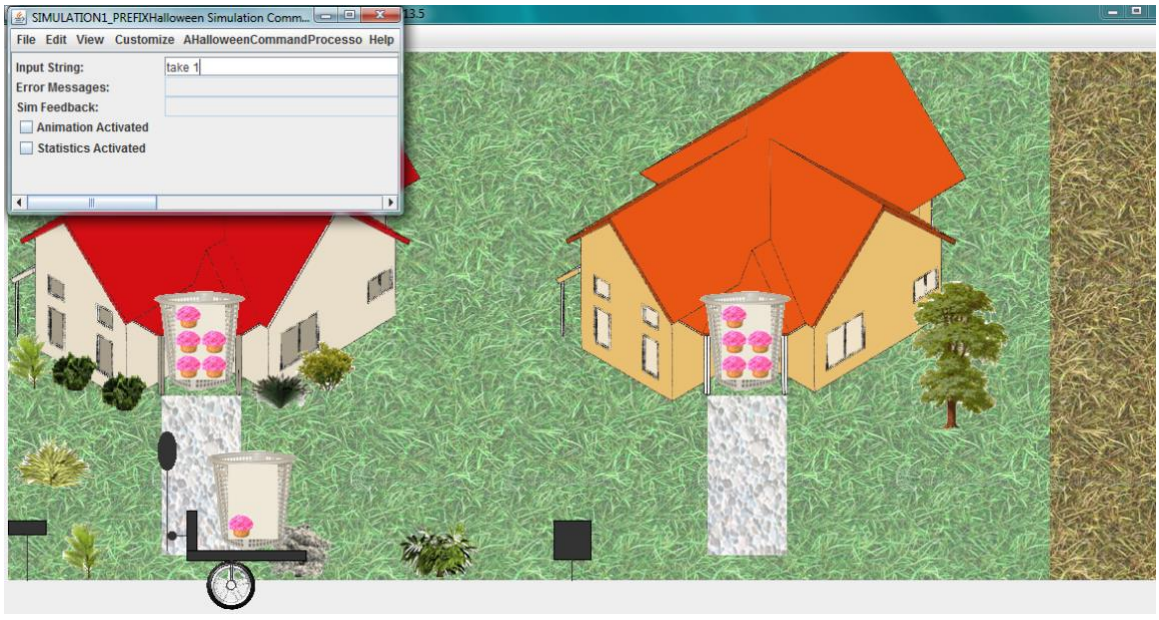
<Undo Command> → undo

<Redo Command> → redo

Try some of these commands, in particular those shown in Figure 1.



(a) Avatar moves into path of leftmost house



(b) Avatar takes one candy

Figure 1 Beau's Non-Distributed Simulation

Intra-Process Replication and Boiler Plate Code

Command Processor for Trapping Changes

The application is built using the MVC design pattern. An instance of the class stringProcessors.AHalloweenCommandProcessor, is the command model that processes user commands. *The command model is the only class relevant to your assignment.*

As mentioned above, you will use Beau's non-distributed simulation in your solution. "All" you need to know about Beau's program is how to trap (using the observer pattern) a command entered by the user and execute such a command programmatically.

Template Code

How this is done is illustrated by the Eclipse Git project, CoupledTrickOrTreat , which serves as a template for this assignment. This code couples two simulations in the same process. This means the same process has two sets of objects for the two simulations, and the process ensures these sets are kept consistent with each other. This coupling is done by ensuring that each command input to the command model in one simulation is also executed by the command model in the other simulation. Your task will be to extend the template to couple simulations in (possibly an arbitrary number of) processes.

Again, this template is best understood if you run the main program of the project.

The main class of the project is coupledsims.TwoCoupledHalloweenSimulations.

Run the template main `TwoCoupledHalloweenSimulations )(` from the 533 library or by creating a main as you did for Beau's project.

Enter commands in the command windows in both simulations and see the effect of each command on both simulations.

Look at the traces (starting with I***) in the coupled simulation program output to illustrate how it links the two command processor models.

Predefined Traces

Traces of the following form are produced before any interaction occurs:

```
I***@449111060378200 {main}(AddedPropertyChangeListener)
EvtSrc(AHalloweenCommandProcessor)
I***@449113271893400 {main}(AddedPropertyChangeListener)
EvtSrc(AHalloweenCommandProcessor)
I***@449113418810500 {main}(AddedPropertyChangeListener)
EvtSrc(AHalloweenCommandProcessor)  coupledsims.ASimulationCoupler@6e70861
I***@449113419215800 {main}(AddedPropertyChangeListener)
EvtSrc(AHalloweenCommandProcessor)  coupledsims.ASimulationCoupler@5fcff272
```

These traces are like the ones you generated in A1-A3. Each of them describes some event of interest. It describes the current thread (e.g. main), the event name (e.g. AddedPropertyChangeListener) the source or class that reported the event (e.g. AHalloweenCommandProcessor), and additional, event-specific parameters (e.g. coupledsims.ASimulationCoupler@14cc1f5c).

Local Observer Registration

In the above trace, the first two steps are taken by UI actions and are not relevant to your assignment (They register view observers, whose toString() methods return the empty strings, with each of the two coupled instances of the model class, AHalloweenCommandProcessor. The main thread executes both calls. These two steps ensure that the observing views are kept consistent with the models.)

Combined In and Out Coupler

The next two steps are relevant to the project. They register two different instances of the class, ASimulationCoupler, as coupled simulations, with two observable simulation models. The task of each of these couplers is to transmit changes in one command-processor model to the other model in the same JVM. In your assignment, you will create client couplers that transmit changes to remote processes and receive changes from them. Moreover, you should break up each coupler into an in-coupler and an out-coupler.

Given two observable simulations S1 and S2, two couplers are created C1 and C2, which observe their observable simulations, S1and S2, respectively, and in turn, update their observing simulations, S2 and S1, respectively. S1 and S2 keep references to C1 and C2, respectively, in their observer lists. C1 and C2 keep references to S2 and S1 in instance variables, called `observingSimulation.`

API for Trapping and Injecting Events in the Simulation

To understand this coupling mechanism better, look at the following method in ASimulationCoupler:

```
	@Override
	public void propertyChange(PropertyChangeEvent anEvent) {
		if (!anEvent.getPropertyName().equals("InputString")) return;
		String newCommand = (String) anEvent.getNewValue();
		LocalCommandObserved.newCase(this, newCommand);
//		System.out.println("Received command:" + newCommand);
		observingSimulation.processCommand(newCommand);
	}
```

This is the method invoked in response to a change made by its observable simulation. The observable simulation invoked this method because the coupler registered itself as an observer of this observable. The command is the value of the InputString property of the observed active simulation.

Thus, the method receives the command executed by the observed simulation in its PropertyChangeEvent parameter. It retrieves the command string from the event and then, in the last statement, calls the processCommand() method to inject the change in its reactive observing simulation .

Thus, each coupler has both an active observable simulation and a reactive observing simulation, which exist in the same process. In your project, this coupler will be split into an in coupler and out coupler as described above.

Event Flow Example Through Traces

Such changes are triggered, of course, by user input and the MVC architecture processes this input to convert it into appropriate output. Let us see the MVC event flow through the traces produced by entering the command, `move 100 -100`, in the first (left) simulation command window:

```
I***{AWT-EventQueue-0}(SetProperty) EvtSrc(AHalloweenCommandProcessor)
InputString = move 100 -100

I***{AWT-EventQueue-0}(CommandSubmitted) EvtSrc(AHalloweenCommandProcessor)
Command:move 100 -100

I***{AWT-EventQueue-0}(NotifiedPropertyChangeEvent)
EvtSrc(AHalloweenCommandProcessor)
java.beans.PropertyChangeEvent[propertyName=InputString; oldValue=;
newValue=move 100 -100; propagationId=null;
source=stringProcessors.AHalloweenCommandProcessor@70c56434]->[,
coupledsims.ASimulationCoupler@3445341]

I***{AWT-EventQueue-0}(LocalCommandObserved) EvtSrc(ASimulationCoupler)
Command:move 100 -100

I***{AWT-EventQueue-0}(CommandSubmitted) EvtSrc(AHalloweenCommandProcessor)
Command:move 100 -100
```

The input results in the execution of a setter in the first command-processor model. As the command-processor model is connected to the (graphics) simulation model, the setter executes the corresponding method (move) in the simulation model. It also notifies its two observers (view and coupling observer) about the set property. The coupling observer submits the command directly to the command processor of the other command-processor rather than setting the InputString property (Why not set the property in the second model?). (In both models, the command execution results in notifications from the simulation model to the views that manage the contents of the graphics window showing the simulation. These steps are not shown.) The steps taken in response to user input are executed, not by the main thread, which has terminated by now, but by the AWT thread created by Java, which processes the user edits of the Input String in the command window.

Main Classes

Here is the code for the main class of coupled simulations:

```
public class TwoCoupledHalloweenSimulations {
	public static void main (String[] args) {
		new AStandAloneTwoCoupledHalloweenSimulations().start(args);
	}

}
```

You will create two such main classes, one for the broadcasting server and one for the client processes.

As you see above, the main class does very little, all it does is instantiate another class, which we will refer to as a *coupled simulation class*, and invoke the start method in it.

Look at the header of the given coupled simulation class:

```
public class AStandAloneTwoCoupledHalloweenSimulations extends
AnAbstractSimulationParametersBean
```

It is an extension of the abstract class assignments.util.inputParameters. AnAbstractSimulationParametersBean (in the 533 jar).
You will create two coupled simulations, one for the server and one for the client. It may be be most convenient to make your coupled simulations subclasses of this predefined class, `AStandAloneTwoCoupledHalloweenSimulations`, though a better approach is to delegate to this class or just copy the code over. Either way the client and server classes must extend (directly or indirectly) the abstract bean parameter class.

A major difference between the server coupled simulation class and the client coupled simulation class is that the server version does not create a copy of the simulation. For this reason, it may be better to make both of them, like AStandAloneTwoCoupledHalloweenSimulations, a subclass directly of AnAbstractSimulationParametersBean .

If you make them subclasses of AStandAloneTwoCoupledHalloweenSimulations then make sure you disable the creation of simulation in the server.

Flexible Port Management and Main Args Processing

As mentioned above, you will convert the standalone coupling program into an RMI-based server and client of a distributed program. As also mentioned above, in this assignment, the RMI registry will be a separate process. As you know from class lectures, a client and server need to know the host and port for the registry host they contact. For flexibility reasons, it should be possible to provide it when the programs are started as main args rather than hardwiring it into the code. Rather than requiring you to write the args processing code yourself, we have provided a library to do this processing. If the expected args are not provided, the library assumes default values.

To activate this and another library you need in A5, A6, you will need to add or inherit the following code in your client and server simulation main classes to process user commands

```
	@Override
	public void  start (String[] args) {
		init(args);
		// register a callback to process actions denoted by the user
commands

	SimulationParametersControllerFactory.getSingleton().addSimulationParame
terListener(this);
		// use the calling back library
```

```
        SimulationParametersControllerFactory.getSingleton().processCommands();


        }
```

If you create your client and server main classes as subclasses of the template main class, you will have this code.

For this assignment, you will need to make the following calls in ServerArgsProcessor:

```
        ServerArgsProcessor.getRegistryHost(args);
        ServerArgsProcessor.getRegistryPort(args);
```

You will need to make the following calls in RegistryArgsProcessor:

```
        RegistryArgsProcessor.getRegistryPort(args)
```

In the ClientArgsProcessor, the following calls are needed:

```
        ClientArgsProcessor.getClientName(args);
        ClientArgsProcessor.getRegistryHost(args);
        ClientArgsProcessor.getRegistryPort(args);
```

Both the client and server need to know the host of the registry process. ClientArgsProcessor and ServerArgsProcessor support calls to get this information. These two as well as the RegistryArgsProcessor support calls to get the registry port. All three processors are in the GIPC package: assignments.util.mainArgs.

Below the ClientArgsProcessor (assignments.util.mainArgs.ClientArgsProcessor) is demonstrated, which is to be invoked by your client program. The server program will use ServerArgsProcessor. We will call these classes the *arg processor* classes.

The coupled simulation class has the following called by the init() call in the start() method inherited from the coupling simulation class:

```
        /**
         * Just printing some of the client args needed for the RMI assignment
by a client.
         * In your client, you will need to use the printed client values in
calls to
         * the RMI API. In addition, in the server part, you will need to call
         * ServerArgsProcessor instead to get the server args.
         */
void processArgs(String[] args) {
        System.out.println("Registry host:" +
                ClientArgsProcessor.getRegistryHost(args));
        System.out.println("Registry port:" +
                ClientArgsProcessor.getRegistryPort(args));
        System.out.println("Server host:" +
                ClientArgsProcessor.getServerHost(args));
```

```
        System.out.println("Headless:" + ClientArgsProcessor.getHeadless(args));
        System.out.println("Client name:" +
                ClientArgsProcessor.getClientName(args));

        // Make sure you set this property when processing args
        System.setProperty("java.awt.headless",
                ClientArgsProcessor.getHeadless(args));

        }
```

This method is called by the main() method of the project, which passes it the main args the user provided.

The result of executing this method is to print the following at the start:

Registry host:localhost
Registry port:1099
Headless:false
Client name:Generic Client:0.5793750661843273

Three of the arg values, relevant to this assignment, are printed above by the coupled simulations program. The program simply retrieves this information from the library, and is oblivious to the order of the arguments or whether the pulled information is default or user-supplied values. *For now, you do not need to supply any main args in your assignment, as you will be running all processes on the same host.* If you run the code on distributed computers in the cloud, the order will become important.

The headless argument determines if the GUI is created. While running tests or when running the program on displayless clients (as in Gradescope), it is important for the Gradescope checks to support disabling of the GUI. As we see, by default, your clients will not be headless.

The client name uniquely identifies a client. The default implementation tries to ensure uniqueness by appending a named constant with a random number.

Default and Test Port Numbers

Ports are important not only for this, RMI assignment, but also for the next two, GIPC and NIO assignments, which extend this assignment. It is important for these assignments to not choose ports that conflict with each other and with ports that are used by your operating system.

The tests on the other hand use a range of ports, defined below:

TEST_PORT_NIO_START = 5000;
TEST_PORT_NIO_END = 6000;
TEST_PORT_RMI_START = 6000;
TEST_PORT_RMI_END = 7000;

TEST_PORT_GIPC_START = 7000;
TEST_PORT_GIPC_END = 8000;

If these ports conflict with the ones your tests use, you can dynamically change the ports the tests use as follows.

In your assignment runner, add the following import:
import gradingTools.comp533.flexible.PortNumbers;
Before running the assignment suite, change the port number:
PortNumbers.setTestPortNIOStart(1100);
 S25Assignment4Suite.main(args);

This may become more relevant for future assignments.

Tracing RMI Calls and Consensus Steps

As in the previous assignments, you will trace important steps in your algorithm for both debugging and localchecks. The tracing code is no longer provided by different methods of an abstract class. Instead, each kind of trace is associated with its own class, which is illustrated in the template code. You need to call the static newCase() method in each of these classes and the first argument, aSource, should be the object calling this method: **this.:**

```
	RMIObjectRegistered.newCase(this, clientName, (ClientProxy)this,
registry);
```

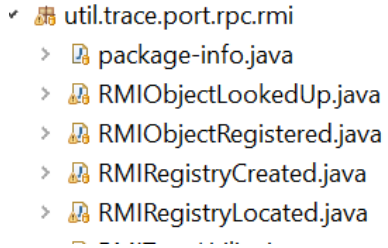
If the newCase() method is called from a static method (not advised) in class , then the first argument should be the class object for that class as in:

```
	RMIObjectRegistered.newCase(ClassContainingStaticMethod.class,
clientName, (ClientProxy)this, registry);
```

In this assignment, you will use trace classes in two packages in GIPC : util.trace.port.rpc.rmi and util.trace.port.consensus.

RMI Calls (util.trace.port.rpc.rmi)

As the RMI library is not under our control, you will need to manually put trace statements to debug and demonstrate your algorithm. The package, util.trace.port.rpc.rmi, in GIPC has the following trace classes with “obvious” semantics based on their names and the signatures of the methods:

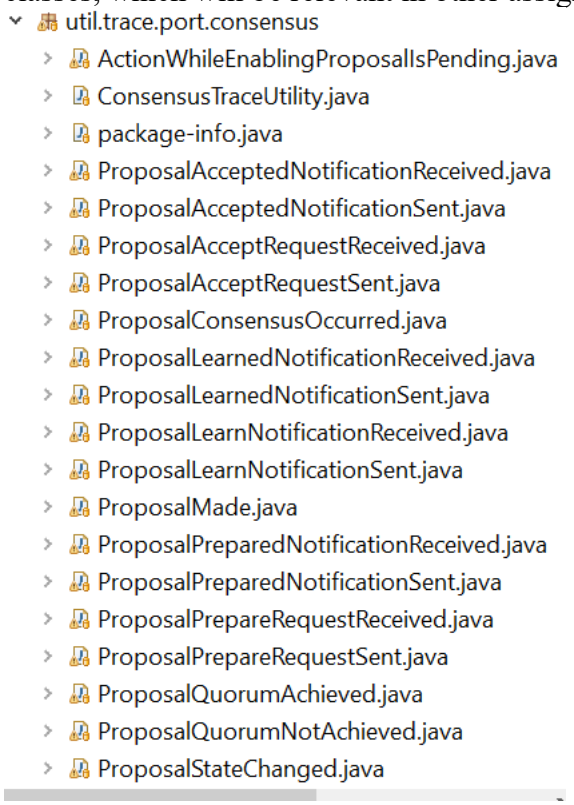


Centralized Asynchronous Consensus (util.trace.port.consensus)

There are many ways to broadcast state changes to other replicas, which is also called the consensus problem. This is the problem of a set of communicating processes, C, agreeing on a value, V, of some replicated object, O. In all forms of consensus, some process proposes a new value of object named O. How other processes react to this proposal varies according to the form of consensus supported. In this assignment, you are implementing centralized asynchronous consensus, which involves the following steps:

1. ProposalMade: A proposer makes proposal V for object O.
2. RemoteProposeRequestSent: The proposer sends a request to change O to a central server, S.
3. RemoteProposeRequestReceived: The server receives the request.
4. ProposalLearnedNotificationSent: The server sends the value to clients other than the one that made the change - members of C who have replicas of O and did not initiate the change. In RMI, this means it simply invokes a "callback" method for notifying the clients of the new value.
5. ProposalLearnedNotificationReceived: The receiving processes receive the value.
6. ProposedStateSet: The receiving processes set their local, replicated, copy to O.

To make you more aware of what you are doing and to automatically grade your assignment, you must trace these steps, using classes in the GIPC package, util.trace.port.consensus. In the above algorithm, the names of the steps are also the names of the classes in the package that trace the step. The package has many other classes, which will be relevant in other assignments:



The classes require the name of the shared object changed. In this assignment, you are changing a single object - the (next) command. Thus, when a user invokes "move 50 -50", its client makes a proposal to change the shared object, next command, to the value "move 50 -50". You should use the named constant util.trace.port.consensus.communication.CommunicationStateNames.COMMAND to identify the changed object.

```
ProposalMade.newCase(this, CommunicationStateNames.COMMAND, -1, event);
```

Later you will be changing two other additional objects describing the meta state kept by the consensus algorithm: the broadcast mode (atomic, asynchronous) and IPC mechanism (RMI, GIPC, NIO). The same interface provides named constants to identify all three shared objects:

```
package util.trace.port.consensus.communication;
public interface CommunicationStateNames {
	public static String COMMAND = "command";
	public static String BROADCAST_MODE = "broadcast_mode";
	public static String IPC_MECHANISM = "ipc_mechanism";
}
```

The newCase() method of the consensus trace classes have mostly "obvious" parameters. The call to the method should be taken **before the step is taken unless the result of the step is a parameter to the step**. In this assignment, the proposal number in all of these calls should be -1. This is illustrated below and the example above:

```
RemoteProposeRequestSent.newCase(this,
	CommunicationStateNames.COMMAND, -1, newCommand);
// code to send the command to the server
```

Here the tracing method is executed before the step is completed.

Enabling Traces

Whether a trace produced by a trace class, T, shows up is determined by two factors. First, whether *all* traces are enabled or not. The trace command (discussed later) controls this Boolean parameter. Second, whether the particular trace class, T, is enabled or not. GIPC includes "trace utility" classes that enable and disable sets of related trace classes. Like the template code, invoke the following snippet at the start of both the client and server:

```
protected void setTracing() {
	PortTraceUtility.setTracing();
	RMITraceUtility.setTracing();
	NIOTraceUtility.setTracing();
	FactoryTraceUtility.setTracing();
	ConsensusTraceUtility.setTracing();
	ThreadDelayed.enablePrint();
	trace(true);
```

```
}
```

Many of these traces are relevant only in future assignments.

Tags

To help the auto grader find the server, client and registry, you will not define a configuration class as in A1-A3. Instead, you will tag the classes by prefixing the class declarations with tagging annotations:

```
import util.annotations.Tags;
import util.tags.DistributedTags;
@Tags({DistributedTags.SERVER, DistributedTags.RMI})
public class <Server Name> …
-----------------------------------------
import util.annotations.Tags;
import util.tags.DistributedTags;
@Tags({DistributedTags.CLIENT, DistributedTags.RMI})
public class <Client Name> …
-----------------------------------------
import util.annotations.Tags;
import util.tags.DistributedTags;
@Tags({DistributedTags.REGISTRY, DistributedTags.RMI})
public class <Registry Class> …
```

The A4 checkstyle file specifies the tag combinations expected in the Tags list.

@DistributedTags.CLIENT+@DistributedTags.RMI,

@DistributedTags.SERVER_REMOTE_INTERFACE+@DistributedTags.RMI,

@DistributedTags.CLIENT_CONFIGURER+@DistributedTags.RMI,

@DistributedTags.CLIENT_REMOTE_INTERFACE+@DistributedTags.RMI,

@DistributedTags.REGISTRY+@DistributedTags.RMI,

@DistributedTags.CLIENT_REMOTE_OBJECT+@DistributedTags.RMI,

@DistributedTags.CLIENT_OUT_COUPLER+@DistributedTags.RMI,

@DistributedTags.SERVER+@DistributedTags.RMI,

@DistributedTags.SERVER_REMOTE_OBJECT+@DistributedTags.RMI,

@DistributedTags.SERVER_CONFIGURER+@DistributedTags.RMI,

The client/server is the program that runs the client/server main. The client/server configurer is the class that configures the connections between the clients and server such as creating and binding remote objects. The client/server remote interface is the interface of the remote object exported by the client/server. The client/server remote object is the class of the client/server remote object. The client out coupler is the object that sends commands from the client to the server. As the specification above indicates, all of these classes must include the RMI tag. Make sure you do not put + in the tag list. Thus, the tag specification DistributedTags.***CLIENT*** + DistributedTags.***RMI*** translates to writing the following code.

```
import util.annotations.Tags;
import util.tags.DistributedTags;
@Tags({DistributedTags.CLIENT, DistributedTags.RMI})
public class <Client Name> …
```

Limiting Console Output

The traces can get so long that they exceed the capacity of your current Console buffer. To ensure that Eclipse does not constrain the size of the buffer, on Windows/Eclipse execute:

Windows→Preferences→Run.Debug→Console (uncheck limit console output)

On Mac it is

Eclipse → Preferences → Run/Debug (drop down arrow in the sidebar) → Console (uncheck limit console output)

Premature Termination of Processes by Checks

The tests use timeouts to address the halting problem and determine if you have infinite loops. To change the timeout seconds, execute a call of the following form in your runtests:

```
Assignment4Suite.setProcessTimeOut(45);
```

Key Issues and Components of Basic Implementation

Read this part only after you have thought thoroughly about your design.

As mentioned several times, you must create a distributed version of this simulation that involves an arbitrary number of simulation processes, with each simulation process implementing a complete user-interface with its own inherited command interpreter (discussed later) and display of the simulation. The simulations created by these processes are coupled similarly to the way the two simulations are coupled in the demo project – after a user submits a command to one simulation, the command should be executed by each of the other simulations. You should use RMI for communication.

Each client will have a replicated copy of the simulation that is kept consistent by relaying messages through the server, which does not have a copy of the simulation. In other words, the initial states of the simulations are the same - hence they are copies - and subsequent states are synchronized through message exchanges. The assignment does not require you to support latecomers who join after interaction begins, though, you are free to, for fun but not extra credit, handle latecomers.

RMI registry is a server for both the server and clients in your application, and must be started before them. The server in your application must, of course, be started before the clients.

Your server will know about all of the client processes and broadcast commands it receives to them. It is possible to also implement something more sophisticated, such as peer-to-peer (P2P) communication (direct communication between client processes), with the server process simply keeping the addresses of the clients. For simplicity reasons, use a server for all parts of the project. A client sends each input command a message to the server, and the server broadcasts that message to the other clients (In the atomic broadcast mode, to be implemented later, it sends it back to the sending client as well).

Your server allows clients to announce themselves, and relays data from clients to other clients. The clients, announce themselves to the server, and send and receive data from the server.

You must design the objects whose methods are invoked remotely. This means you must do extra work to make your program more modular. You must create object proxies in both the server and client to communicate data.

The proxy that resides in the server will be registered with RMIRegistry and the clients can be aware of its name. However, the clients will not be known to the server, which should know no details of the clients. The proxies in the clients should be passed as callback objects in calls made in the server.

Let the server export and register an object - the relayer object - that provides a join method called by a client to associate its client name (retrieved from

ClientArgsProcessor) with a callback client exported object. It also includes a broadcast method called by a client to send a command to other clients.

The broadcast method needs to know not only the command to be echoed but also the identity of the calling client to ensure it avoids an echo back to the client, which can result in an infinite loop. This should be the client name passed to the join method.

Ideally, a client should be able to export the command processor object (provided by the non-distributed simulation) directly to the server, which can remotely set the next command to it. However, RMI requires classes of exported objects to be distribution-aware – they must implement the Remote interface. Thus, in your client, you will need to create and export a distribution-aware programmer-defined "proxy" for the command processor object in Beau's non-distributed simulation. The server will use an RMI-generated proxy to call a method in this programmer-defined proxy to call a method in the non-distributed simulation.

Workflow: A Student's Viewpoint

Read this if you are still confused. The following conversation between two students may make the task workflow more understandable. This conversation is outdated as the tracing in this assignment has changed.

Issue from a student: I have read all this and still don't know where to start. (Chris's Response)

1. Registry
    1. Copy registry from Dewan's example
    2. Locate registry in client and server
2. Proxies
    1. Create server proxy class
        1. Export server proxy class in server
        2. Bind server proxy (use some final static name and hardcode it in client and server)
    2. Create client command process proxy class
        1. Export command process proxy
        2. Register it with the server with the connect/registration command you wrote
    3. Implement methods in Server proxy to broadcast commands
    4. Implement methods in Client proxy to receive commands
    5. Implement method in Server proxy to allow clients to register once they get the proxy from the registry
        1. Create a list of clients who've registered in the server proxy
        2. After writing the method, use the method in the client class to register after it gets the server proxy
3. Subclass SimulationParameterBean (a modification of the template subclass)
    1. Client and server get their states from the inherited bean class.
4. Traces in assignment doc

1. Read Prof Dewan's section on the traces:
    1. ProposalMade
        1. In Client and server simulationParameterBean
    2. RemoteProposeRequestSent
        1. All in client SimulationParameterBean
        2. Also server simulationParameterBean for setting atomic/broadcasting state
    3. RemoteProposeRequestReceived
        1. All in server broadcaster proxy
    4. ProposalLearnedNotificationSent
        1. All in server broadcaster proxy
    5. ProposalLearnedNotificationReceived
        1. All in client command proxy
    6. ProposedStateSet
        1. Command state set in client command proxy
2. Registry traces: util.trace.port.rpc.rmi
    1. Read Prof Dewan's section
        1. Create (WIP)
        2. Located when located in server and client
        3. Registered when binding exported objects in server and clientLookup when getting server proxy in client

Assignment Overview: GPT's Point of View

Here is a response to my following GPT query:

> *I have written a distributed assignment for a class. I need help with clarifying confusing parts and creating summaries of sections*.

Below is its response, which I have edited a bit because it gave away the proxy interfaces.

Before you read the response, you may find the following conversation with it about proper English interesting.

> **You said:**
> I am curious, you used the term, broadcasted above. Is that good or even correct English? Should it not be broadcast instead?
> **ChatGPT said:**
> Good catch! The correct past tense of **broadcast** is simply **broadcast** (not "broadcasted").
> - **Correct:** "The message was **broadcast** to all clients."
> - **Incorrect:** "The message was **broadcasted** to all clients."

"Broadcast" is an **irregular verb**, meaning its past tense remains the same as its base form, similar to **cut, put, or set**.

I have often wondered how and if it separates good and bad English/code. I think in this case, the programmers who write comments tend to make the past tense of this irregular verb different from the present tense.

Here is GPT's edited response.

Summaries of Key Sections

1. Overview of the Assignment

- **Goal**: Implement **object replication** in a distributed system using **Java RMI**.
- **Context**: You'll be making a **Halloween Trick-or-Treat simulation** collaborative across multiple clients.
- **How?**:
  - Start with a **single-user simulation**.
  - Extend it into a **distributed, replicated version**.
  - Use **remote procedure calls (RMI)** to communicate state changes between replicas.

---

2. System Architecture & Design

- **Replication Concept**:
  - **Each client keeps its own simulation copy** (replica).
  - **Changes from one client are broadcasted** to others via a **central server**.
  - **Main components**:
    - **Out-coupler**: Sends state changes from one client.
    - **In-coupler**: Receives state changes and updates the local replica.
    - **Relayer (Server-side component)**: Distributes state changes.
- **Workflow**:
  - **Client modifies state** → Sends to **server relayer**.
  - **Server relayer broadcasts** to other clients.
  - **Clients update their replicas** to match the new state.

---

3. Setting Up the Non-Distributed Simulation

- Provided as **compiled code (JAR file)**.
- Uses **images and graphical elements**.
- **Commands to interact**:
  - move x y → Moves the avatar.
  - take n → Takes candy from a house.
  - give n → Gives candy to a house.
- Goal: **Understand how the standalone simulation works before making it distributed**.

---

4. Implementing Distributed Replication

- **Steps**:

1. **Understand intra-process replication**: The provided CoupledTrickOrTreat project shows how two simulations can stay in sync **within the same process**.
2. **Extend this to inter-process replication**:
    - **Use RMI** to sync simulations across different processes.
    - **Implement out-couplers (sends commands) and in-couplers (receives commands).**
    - **Clients register with the server**, which keeps track of them and forwards updates.

- **Technical Requirements**:
    - Clients communicate **only through the server**.
    - **Clients do not register with RMI Registry** (only the server does).
    - **Use tracing tools** to debug execution flow.

---

5. RMI Communication Details

- **Main Remote Methods**:
    - **Clients**:
        - Call a registration method in the server to connect.
        - Call a command-sending-method in the server to relay the command.
    - **Server**:
        - Calls a broadcasting mehod to notify all clients except the sender.
        - Clients update their own state upon receiving a broadcast.
- **Registry & Startup Order**:
    - Start **RMI registry**.
    - Start **server** (registers remote object).
    - Start **clients** (connect to the server).

---

6. Debugging & Tracing

- **Tracing required for auto-grading**:
    - When a command is sent.
    - When the server processes it.
    - When clients receive the update.
- **Console Logs**:
    - Ensure logs don’t exceed buffer size.
    - Adjust Eclipse settings if necessary.

Clarifications for Potentially Confusing Parts

1. The Role of the Server

Potential Confusion: The assignment says the server "does not store a simulation replica." What does that mean?

Clarification:

- **The server does NOT have a local game simulation.** It only forwards updates from one client to the others.
- Clients **register with the server** and send their updates to it.
- The **server’s job is just to relay updates**—not to maintain any game state.

---

2. Why Split In-Coupler and Out-Coupler?
Potential Confusion: Why can't a client just send updates directly to other clients?
Clarification:

- This is a **centralized system**, meaning:
  - All communication **must pass through the server**.
  - Each client **only sends updates to the server**, not directly to other clients.
- **Splitting the couplers** makes it clear:
  - The **out-coupler** sends updates to the server.
  - The **in-coupler** receives updates from the server and applies them.

---

3. Understanding the Tracing Requirement
Potential Confusion: Why are traces necessary?
Clarification:

- Traces help **debug** and **prove correctness**.
- You **must log key events**, such as:
  - When a client sends a command (ProposalMade).
  - When the server processes it (RemoteProposeRequestReceived).
  - When clients update (ProposalLearnedNotificationReceived).

---

4. Registering Clients Without the RMI Registry
Potential Confusion: Why don't clients register with the RMI registry?
Clarification:

- **Only the server registers its remote object** in the RMI registry.
- Clients **look up the server in the registry** but **register themselves directly with the server**.
- This allows the server to track connected clients dynamically.

Final Thoughts

- **Key Takeaway:** This assignment is about understanding **replication** and **RMI-based communication**.
- **Main Challenges**:
  - **Understanding how to forward updates via the server**.
  - **Properly implementing the in-coupler/out-coupler model**.
  - **Ensuring clients register and communicate correctly**.
- **Recommended First Steps**:
  0. Run the **non-distributed simulation** to understand it.
  1. Run the **CoupledTrickOrTreat project** to see how local coupling works.
  2. Implement **client-server communication** with a **minimal RMI example** before integrating it into the project.

Advanced Concepts for Future Assignments

To better understand the code with which you are working and prepare yourself for the next assignments, here is some material, at which you should probably look after you have made progress on the assignment.

Halloween Simulation API

As mentioned above, you will also use Beau's standalone simulation in your solution. The discussion above describes how the template code must be adapted. This section helps you understand better what the template code does.

This project uses public methods in the following interface in package stringProcessors in Beau's project:

```
public interface HalloweenCommandProcessor{ … }
```

A relevant method in this interface is:

```
void processCommand(String newInputString);
```

This method asks the simulation to invoke the command, newInputString, on the local simulation. In your assignment, this method must be invoked by an in-coupler on the simulation in response to commands executed in other, remote, simulation processes.

Another relevant method in the interface is:

```
void addPropertyChangeListener(PropertyChangeListener newListener);
```

It can be used by an out-coupler to listen to or trap commands entered by a user by registering an instance of predefined Java type java.beans.PropertyChangeListener. The simulation notifies each registered instance of the new command by calling the method:

```
void propertyChange(PropertyChangeEvent evt)
```

in the instance. The new command is announced as the new value of the property name, "InputString". New values of this property are needed to send user commands to remote processes.

The two other important methods in this interface are:

```
boolean isConnectedToSimulation();

void setConnectedToSimulation(boolean connectedToSimulation);
```

An instance of the command processor can be created using the static method BeauAndersonFinalProject.createSimulation() – its use is illustrated in the coupled simulation project.

```
protected HalloweenCommandProcessor createSimulation1(String aPrefix) {
      return
         BeauAndersonFinalProject.createSimulation(
                        aPrefix,
                        coupledsims.Simulation1.SIMULATION1_X_OFFSET,
                        coupledsims.Simulation.SIMULATION_Y_OFFSET,
                        coupledsims.Simulation.SIMULATION_WIDTH,
                        coupledsims.Simulation.SIMULATION_HEIGHT,
                        coupledsims.Simulation1.SIMULATION1_X_OFFSET,
                        coupledsims.Simulation.SIMULATION_Y_OFFSET);
}
```

Each client will create a single simulation, so you have to worry about offsets, width, height of only one simulation.

Study the template code well before you start making changes.

Processing of Interactive Parameters (assignments.util.inputParameters*)*
Processing of interactive commands should be optional for this assignment. However, you may want to read about them if you want to interact with the simulation only through the console, without using the GUIS.

As you know, in any OS, it is possible to use a GUI or a command line interface to manipulate OS objects. We saw above the Fig. 1 how one can execute commands such as *move* and *take* from a GUI.

As we saw in A1-A3, a console-based user-interface is convenient and, more important, necessary for testing. Such an interface is provided by library code to manipulate the simulation and change parameters of the distributed algorithm you will build in A4-A6. Both the simulation commands and the algorithm parameters are called parameters by the library code. These are received by registering with the library code a SimulationParameterListener, shown in the third statement in the start() method below:

```
	@Override
	public void start(String[] args) {
		setTracing();
		processArgs(args);
		// register a callback to process actions denoted by the user
commands

	SimulationParametersControllerFactory.getSingleton().addSimulationParame
terListener(this);
		// use the calling back library
	SimulationParametersControllerFactory.getSingleton().processCommands();

	}
```

Most of the methods of SimulationParametersListener (except quit(), simulationCommand(), and experimentalInput()) are expected to involve setting of some state. The abstract class your main classes inherit provides getters and setters for the changed dynamic parameter state, and automatically calls the setters of this state from the corresponding parameter changing input. For instance, when the user inputs the "trace true" command, the abstract class calls a setter to change a instance variable determining if the traces should be displayed, which can then be retrieved by a getter.

You can instantiate your two coupled simulation subclasses (for client and server) of this abstract class as singletons accessible from a factory, which can then be accessed globally from different components of your code to access the parameter state (by calling the getters()).

Invoking and Trapping Interactive Commands

Let us interact with the template simulation to understand the nature of provided console commands and invoked parameter bean method to trap these commands.

s(imulationCommand) String

s(imulationCommand) takes a string argument. The text in parentheses is optional when inputting this command.
The following console excerpt shows its use in the template code:

```
Enter (prefixes of) print, quit or a method and its parameters separated by
commas:
s "move 50 -50"
public default void
assignments.util.inputParameters.SimulationParametersListener.simulationComman
d(java.lang.String)[move 50 -50]
I***@1517467216684900 {main}(SetProperty)
EvtSrc(Your_Halloween_Processor_Class_Name)  InputString = move 50 -50
…
I***@1517467248109000 {main}(CommandSubmitted)
EvtSrc(Your_Halloween_Processor_Class_Name)  Command:move 50 -50
```

As mentioned above, the command library processes each input command by invoking a parameter bean method that can be overridden. Before doing so, it prints the header of the method as the first line after the command entry.

The template coupled simulation coupler has overridden the method to process this command as follows:

```
	public void simulationCommand(String aCommand) {
		long aDelay = getDelay();
		if (aDelay > 0) {
			ThreadSupport.sleep(aDelay);
		}
		commandProcessor1.setInputString(aCommand); // all commands go to
the first command window
	}
```

This method sets the input string in the model managing the left command window. Setting the string of course, has the effect of changing both simulations in this code.

Thus, this command provides an alternative way to set this string, which does not require the use of a GUI.

In your implementation, each client will have only one command model, and your implementation will set the input string in this model. Its effect should be to change the simulation in all clients of the server you will implement.

t(race) boolean

This command calls the following code in the coupled simulation:

```
@Override
public void trace(boolean newValue) {
	super.trace(newValue);
	Tracer.showInfo(isTrace());
}
```

The purpose of this command is to dynamically enable and disable tracing as shown below:

```
t false
public default void
assignments.util.inputParameters.SimulationParametersListener.trace(boolean)[f
alse]
I***Tracer: showInfo = false
Enter (prefixes of) print, quit or a method and its parameters separated by
commas:
s "undo"
public default void
assignments.util.inputParameters.SimulationParametersListener.simulationComman
d(java.lang.String)[undo]
Enter (prefixes of) print, quit or a method and its parameters separated by
commas:
t true
public default void
assignments.util.inputParameters.SimulationParametersListener.trace(boolean)[t
rue]
I***Tracer: showInfo = true
Enter (prefixes of) print, quit or a method and its parameters separated by
commas:
s "redo"
public default void
assignments.util.inputParameters.SimulationParametersListener.simulationComman
d(java.lang.String)[redo]
I***@1517547132355200 {main}(SetProperty)
EvtSrc(Your_Halloween_Processor_Class_Name)  InputString = redo
I***@1517547132706400 {main}(CommandSubmitted)
EvtSrc(Your_Halloween_Processor_Class_Name)  Command:redo
I***@1517547143406800 {main}(NotifiedPropertyChangeEvent)
EvtSrc(AHalloweenCommandProcessor)
java.beans.PropertyChangeEvent[propertyName=InputString; oldValue=undo;
newValue=redo; propagationId=null;
source=stringProcessors.AHalloweenCommandProcessor@254e9709]->[,
coupledsims.ASimulationCoupler@4250b0c1]
I***@1517547151180300 {main}(LocalCommandObserved) EvtSrc(ASimulationCoupler)
Command:redo
I***@1517547151424400 {main}(CommandSubmitted)
EvtSrc(Your_Halloween_Processor_Class_Name)  Command:redo
```

The first invocation of the command sets tracing to false. As a result, we do not see any traces when we execute “s undo”. The second invocation sets tracing to true. As a result, we do see traces when we execute “s redo”.

You can directly use the implementation of the trace method in your client and server. This will be most easily done by inheriting the template coupled simulation class.

(delaySends) int

This command calls the following overriding method in the coupled simulation

```
@Override
	/*
	 * This override is not really needed, provided here to show that this
method
	 * exists.
	 */
	public void delaySends(int aMillisecondDelay) {
		// getDelay can be used to determine the delay
		// in other parts of the program
		super.delaySends(aMillisecondDelay);
	}
```

As we will see, network delays can promote inconsistencies. The intention of this command is to simulate such delays. Once the delay is set (interactively) via this command, each call to getDelay() (inherited from the abstract parameter bean class) will return the delay set by this command, as shown below:

```
	/*
	 * You will need to delay not command input but sends
	 */
	public void simulationCommand(String aCommand) {
		long aDelay = getDelay();
		if (aDelay > 0) {
			ThreadSupport.sleep(aDelay);
		}
		commandProcessor1.setInputString(aCommand); // all commands go to
the first command window
	}
```

You will call getDelay() (inherited from the parameter bean class) to delay sending of the command to remote processes via the broadcast server. Your implementation of `simulationCommand()`in either the client or server coupled simulation should not delay commands Thus, you should override this method in your client and server if you set the delay to more than zero.

(experimentalInput)

An important issue in distributed computing, of course, is performance. The absolute performance is not as important as the relative performance of different design decisions. Such performance requires repeating a set of actions under different alternatives. This command is expected to trigger such a repetition. It calls the
following method:

```
@Override
public void experimentInput() {
	long aStartTime = System.currentTimeMillis();
	PerformanceExperimentStarted.newCase(this, aStartTime,
NUM_EXPERIMENT_COMMANDS);
	boolean anOldValue = isTrace();
	this.trace(false);
	for (int i = 0; i < NUM_EXPERIMENT_COMMANDS; i++) {
		commandProcessor1.setInputString(EXPERIMENT_COMMAND_1);
		commandProcessor1.setInputString(EXPERIMENT_COMMAND_2);
	}
	trace(anOldValue);
	long anEndTime = System.currentTimeMillis();
	PerformanceExperimentEnded.newCase(this, aStartTime, anEndTime,
anEndTime - aStartTime, NUM_EXPERIMENT_COMMANDS);

}
```

```
Enter (prefixes of) print, quit or a method and its parameters separated by
commas:
e
public default void
assignments.util.inputParameters.SimulationParametersListener.experimentInput(
)[]
I***@1517562909275000 {main}(PerformanceExperimentStarted)
EvtSrc(AStandAloneTwoCoupledHalloweenSimulations)  1582389387721:500
I***Tracer: showInfo = false
I***Tracer: showInfo = true
I***@1517571674764800 {main}(PerformanceExperimentEnded)
EvtSrc(AStandAloneTwoCoupledHalloweenSimulations)  1582389387721-
1582389396487=8766:500
```

The template implementation executes 500 simulation commands and prints the time required to process these commands. While doing so, it disables tracing, which can not only blow up the console output but also significantly increase the processing time. You can use this implementation directly in your client coupled simulation. If you do not disable traces, you will get a message complaining about too many traces.

(localProcessingOnly)

You will need to compare the performance overhead of your implementation. You can do this by running a client under two conditions:

(1): Local Processing Only: A command undergoes only local processing. In this mode, your client behaves like the original simulation you extended – it does not transmit any message in response to a command. It can react to incoming commands, or not - your decision.

(2): Regular Processing: Each command is processed not only locally but also broadcast.

This command makes no sense in the coupling template, but to illustrate how you should implement it, the template includes an implementation that dynamically removes (add) the coupling between the two command models when local processing only is true/false:

```
@Override
public void localProcessingOnly(boolean newValue) {
    super.localProcessingOnly(newValue);
    if (isLocalProcessingOnly()) {
       commandProcessor1.removePropertyChangeListener(simulation2Coupler);

       commandProcessor2.removePropertyChangeListener(simulation1Coupler);
    } else {

       commandProcessor1.addPropertyChangeListener(simulation2Coupler);

       commandProcessor2.addPropertyChangeListener(simulation1Coupler);
     }
}
```

Here is an interactive invocation of the code above:

```
l true
public default void
assignments.util.inputParameters.SimulationParametersListener.localProcessingO
nly(boolean)[true]
I***@1517598325259700 {main}(RemovedPropertyChangeListener)
EvtSrc(Your_Halloween_Processor_Class_Name)
coupledsims.ASimulationCoupler@4250b0c1
I***@1517598325621100 {main}(RemovedPropertyChangeListener)
EvtSrc(Your_Halloween_Processor_Class_Name)
coupledsims.ASimulationCoupler@7b5e5fa
```

You will thus have to change the implementation of this method in your client and server implementations. The server version will do nothing while the client version will behave as described above.

(atomicBroadcast) boolean

This command will toggle atomic broadcast mode, which is not relevant for this assignment.

This is the template implementation:

```
@Override
/**
* Relevant in consistency assignments only
*/
public void atomicBroadcast(boolean newValue) {
       super.atomicBroadcast(newValue);
       commandProcessor1.setConnectedToSimulation(!isAtomicBroadcast());
       commandProcessor2.setConnectedToSimulation(!isAtomicBroadcast());
}
```

Here is an invocation of this code interactively:

```
a true
public default void
assignments.util.inputParameters.SimulationParametersListener.atomicBroadcast(
boolean)[true]
I***@1586186059033100 {main}(ProposalMade)
EvtSrc(AStandAloneTwoCoupledHalloweenSimulations)  broadcast_mode,-1.0=true
I***@1586186059533800 {main}(ProposedStateSet)
EvtSrc(AStandAloneTwoCoupledHalloweenSimulations)  broadcast_mode,-1.0=true
I***@1586186059986400 {main}(ConnectedToSimulation)
EvtSrc(AHalloweenCommandProcessor)  Connected To Simulation:false
I***@1586186062875600 {main}(ConnectedToSimulation)
EvtSrc(AHalloweenCommandProcessor)  Connected To Simulation:false
```

The command:

a true

removes (establishes) the linkage (implemented by Beau's code) between the two command and (graphics) simulation models. When you address atomic broadcast, you will have only one command and simulation model, and you will change the linkage between the two.

When there is no such linkage, inputting a command does lead to appropriate property notifications but does not affect the graphics simulation. This means a property change of the input string does not result in a command being submitted to the simulation, as we see below.

```
s "undo"
public default void
assignments.util.inputParameters.SimulationParametersListener.simulationComman
d(java.lang.String)[undo]
I***@1585376782452800 {main}(SetProperty)
EvtSrc(Your_Halloween_Processor_Class_Name)  InputString = undo
I***@1585376782960100 {main}(NotifiedPropertyChangeEvent)
EvtSrc(Your_Halloween_Processor_Class_Name)
java.beans.PropertyChangeEvent[propertyName=InputString; oldValue=move 50 -50;
newValue=undo; propagationId=null;
source=stringProcessors.AHalloweenCommandProcessor@3348c987]->[,
coupledsims.ASimulationCoupler@23a9400c]
I***@1585376792083700 {main}(LocalCommandObserved)
EvtSrc(Your_Coupler_Class_Name)  Command:undo
```

Again, this is not relevant for this assignment.

(quit) integer

This command calls the quit method with an integer:

q 0

public default void

assignments.util.inputParameters.SimulationParametersListener.quit(int)[0]

The template implementation calls System.exit() with the passed value. This is what your implementation should also do.

```
@Override
public void quit(int aCode) {
	System.exit(aCode);
}
```

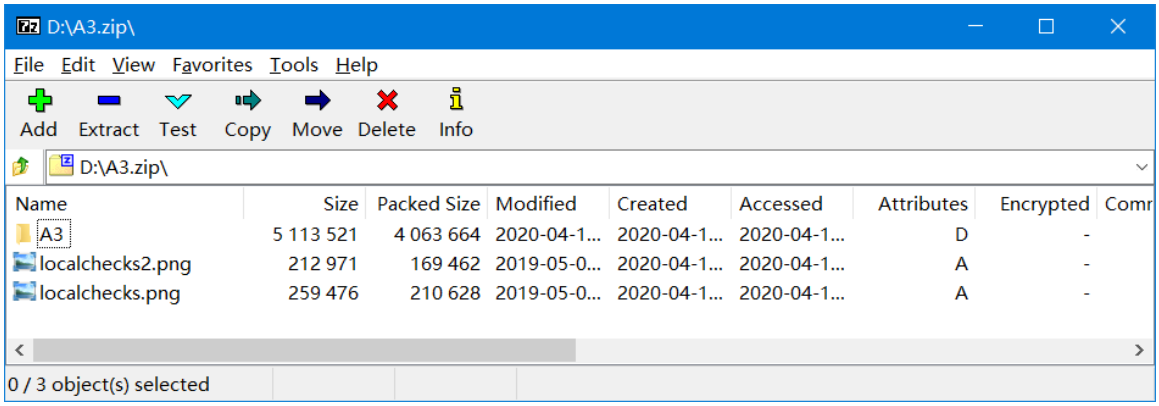
This is the only command a server processes. A client processes all of the commands listed above. This means the server does not run the simulation and its task is to connect the replicas of the clients.

Submission Instructions

As always, upload your code to Gradescope. Do not zip GIPC or Beau's TrickOrTreat project in case you have downloaded them from GIT - we will use our versions of them in the class path when we grade your code. Submit the entire Eclipse project just as in previous assignments.

As in A1..A3 submit screenshots of localchecks results, and also screenshots to convince us (without running localchecks) that your assignment works. These screenshots, as in A1..A3, can simply show console I/O and can be in the spirit of the traces shown above to illustrate the working of various interactive commands.

When zipping your submission, put your screenshots *outside the project folder*, select your project folder and your screenshots, and zip them. Do not put your screenshots inside your project folder, that is where all the images for the simulations are. Your zip file should look like this:



Of course you will have additional image files outside the project.

Possible Problems and Solutions from Past Students (Feel free to add more)

Some of these are outdated.

1. Class with some tag not found.

    a. Make sure the tags are over before the class declaration rather than in it.
2. Processes connect when started manually but not when started from the grader
    a. Make sure the main args for host and port are used rather than hardwiring your program to localhost and default ports.
3. Passed the two client test suite but got the following exception in the one client test suite: “Couldn't run code:Tags:[Server, RMI, NIO]not found”.
4. Traces in static methods: ?”The instructions say we are supposed to use RMIRegistryLocated.newCase(.....) with "this" as the first argument to trace every time we locate the registry. I believe this should be done in a static main method used to create either the server or client proxies, but these methods do not allow you to pass "this" as the first argument because they are static. Is there another place where this should be done instead?” Pass the class object in these methods: e.g. ClassContainingSttaicMethod.class.It is better though if the main method calls a method in an instance that does the registration.
5. Catching echoing of commands: “Do we need to implement PropertyChangeListener/allowPropertyChange for the code to work properly? It seems like it would be much easier to just add a tag to every command we broadcast, then drop incoming ones we've seen before.” Your server should not be echoing back the command to the sender. It should keep track of the sender.
6. Too many traces: “In A4, it seems that when executing 500 commands for the distributed 3-User test will cause TooManyTracesException. Should we just catch that?” This is what the assignment says: The template implementation executes 500 simulation commands and prints the time required to process these commands. While doing so, it disables tracing, which can not only blow up the console output but also significantly increase the processing time. You can use this implementation directly in your clients.
7. .Simulation start() method is blocking, which means anything after start() in the Server/Client launcher will never be executed. Please make sure any remote objects are created, are passed into simulation Class, and are exported before start() method, instead of creating them in the start() method, getting them by Server/Client launcher and then being exported.
8. The grader runs in headless mode. Make sure that your implementation is able to process input provided from the console. A good way to test this is by inputting s “move 10 10”. If your program is not receiving input from the console check the start method in the template code and make sure you have the property change methods in your implementation.